Paper Search v1 定稿文档

1. 背景

当前已有 `arxiv_search`，但它不能作为唯一论文搜索入口。

主要问题：

```
1. arXiv API 有严格限流。
2. arXiv 是预印本来源，不等同于 publisher metadata。
3. arXiv metadata 主要是英文，中文 query 很容易 0 结果。
4. 单一来源失败时，模型容易误判“没有论文”。
```

arXiv legacy API 要求 **最多每 3 秒 1 次请求，并且一次只保持一个连接**，所以必须把它作为低频预印本来源，而不是唯一搜索主力。(arXiv 信息)

---

2. 最终目标

新增统一论文搜索工具：

```
paper_search
```

用于替代模型直接依赖 `arxiv_search` 的行为。

`paper_search` 只使用免费开放来源：

```
1. Crossref
2. arXiv
3. DataCite
4. Unpaywall
```

其中：

```
Crossref   = 出版物 / DOI / publisher metadata
arXiv      = 预印本
DataCite   = DOI / dataset / research object / preprint metadata
Unpaywall  = DOI -> OA link / PDF link 后处理
```

不接入：

```
OpenAlex
Semantic Scholar
```

```
PubMed
Europe PMC
DOAJ
OpenCitations
Scopus
Web of Science
Dimensions
Elsevier / Springer / IEEE / ACM 付费 API
```

---

3. 核心原则

3.1 免费开放

不接入任何:

```
付费 API
商业数据库
freemium quota API
免费额度用完后付费的 API
```

3.2 低并发

允许:

```
Crossref / arXiv / DataCite 多来源并发
```

禁止:

```
单一来源内部并发
多页并发
cursor 并发
Unpaywall 多 DOI 并发
为了性能拉高请求速率
```

第一版每个 source 每次 search 只发 **1 个请求。**

3.3 SDK / 成熟客户端优先

```
arXiv 使用 arxiv Python 包，避免手写 Atom XML 解析。
Crossref / DataCite / Unpaywall 使用官方 REST API 薄 adapter。
```

原因:

```
1. Crossref / DataCite / Unpaywall 的第一版调用很简单。
2. 不引入复杂客户端抽象。
3. 不做复杂二次解析。
4. 只解析必要稳定字段。
```

DataCite REST API 明确支持检索、查询和浏览 DOI metadata records。(DataCite Support) Unpaywall API 请求必须带 email 参数。(Unpaywall) Crossref 建议使用 `mailto` 参数进入 polite pool，且需要处理 429 / rate limit。(www.crossref.org)

---

4. 来源策略

4.1 Crossref

职责:

```
DOI
publisher metadata
bibliographic metadata
期刊 / 会议出版信息
```

启用方式:

```
默认启用
```

调用方式:

```
GET https://api.crossref.org/works
```

参数:

```
query.bibliographic = query
rows = min(rows, 5)
select = DOI,title,author,issued,container-title,URL,abstract,published-print,published-online
mailto = TOOL_CONTACT_EMAIL if present
```

限制:

```
每次 search 只发 1 个请求
不做 cursor
不做多页
不做并发
429 时读取 Retry-After 或指数退避
```

4.2 arXiv

职责:

```
预印本
AI / CS / 数学 / 物理等领域 preprint
```

启用方式:

```
默认启用
```

实现方式:

```
使用 arxiv Python 包
```

依赖:

```
uv add arxiv
```

配置:

```
ARXIV_MIN_INTERVAL_SECONDS = 3.0
ARXIV_MAX_RESULTS = 5
```

限制:

```
每次 search 只执行一次 arxiv.Search
max_results <= 5
不做多页
不做并发
使用 arxiv.Client(delay_seconds=3)
```

4.3 DataCite

职责:

```
DOI metadata
dataset
research object
preprint metadata 补强
```

启用方式:

默认启用

调用方式:

```
GET https://api.datacite.org/dois
```

参数:

```
query = query
page[size] = min(rows, 5)
```

限制:

```
每次 search 只发 1 个请求
不做分页
不做并发
429 时读取 Retry-After 或指数退避
```

4.4 Unpaywall

职责:

```
DOI -> open access link
DOI -> PDF link
```

启用方式:

```
只有 TOOL_CONTACT_EMAIL 存在时启用
```

调用方式:

```
GET https://api.unpaywall.org/v2/{doi}?email={TOOL_CONTACT_EMAIL}
```

限制:

```
只做 DOI 后处理
不作为主搜索 source
最多处理前 5 个 DOI
串行处理
不并发
TOOL_CONTACT_EMAIL 缺失时跳过并 warning
```

---

5. 配置项

```
from typing import Optional
from pydantic import BaseModel



class PaperSearchSettings(BaseModel):
    PAPER_SEARCH_TIMEOUT_SECONDS: float = 12.0
    PAPER_SEARCH_MAX_RESULTS: int = 8
    PAPER_SEARCH_CACHE_TTL_SECONDS: int = 3600

    PAPER_SEARCH_ENABLE_CROSSREF: bool = True
    PAPER_SEARCH_ENABLE_ARXIV: bool = True
    PAPER_SEARCH_ENABLE_DATACITE: bool = True
    PAPER_SEARCH_ENABLE_UNPAYWALL: bool = True

    CROSSREF_BASE_URL: str = "https://api.crossref.org"
    DATACITE_BASE_URL: str = "https://api.datacite.org"
    UNPAYWALL_BASE_URL: str = "https://api.unpaywall.org"

    SOURCE_RATE_LIMIT_SECONDS: float = 1.0

    ARXIV_MIN_INTERVAL_SECONDS: float = 3.0
    ARXIV_MAX_RESULTS: int = 5

    TOOL_CONTACT_EMAIL: Optional[str] = None
```

明确不要加：

```
PAPER_SEARCH_ENABLE_OPENALEX = False
PAPER_SEARCH_ENABLE_SEMANTIC_SCHOLAR = False
PAPER_SEARCH_ENABLE_PUBMED = False
PAPER_SEARCH_ENABLE_EUROPE_PMC = False
PAPER_SEARCH_ENABLE_DOAJ = False
PAPER_SEARCH_ENABLE_OPENCITATIONS = False
```

---

6. 目录结构

```
chat/application/paper_search/
  __init__.py
  models.py
  service.py
  formatting.py
```

```
  query.py
  dedup.py
  ranking.py
  cache.py
  rate_limit.py
  http.py
  sources/
    __init__.py
    crossref_source.py
    arxiv_source.py
    datacite_source.py
    unpaywall_source.py

chat/application/tools/
  paper_search_tool.py
```

---

7. 数据模型

```
from dataclasses import dataclass, field
from typing import Optional


@dataclass(frozen=True, slots=True)
class PaperSearchRequest:
    query: str
    max_results: int = 8


@dataclass(frozen=True, slots=True)
class PaperSearchResult:
    title: str
    authors: list[str] = field(default_factory=list)
    year: Optional[int] = None
    abstract: Optional[str] = None
    venue: Optional[str] = None
    doi: Optional[str] = None
    arxiv_id: Optional[str] = None
    url: Optional[str] = None
    pdf_url: Optional[str] = None
    source_urls: list[str] = field(default_factory=list)
    source_names: list[str] = field(default_factory=list)
    publication_date: Optional[str] = None
    is_open_access: Optional[bool] = None
    result_type: Optional[str] = None
    relevance_score: float = 0.0
```

```
    authority_score: float = 0.0


@dataclass(frozen=True, slots=True)
class PaperSourceResponse:
    source_name: str
    results: list[PaperSearchResult]
    warnings: list[str] = field(default_factory=list)
    failed: bool = False


@dataclass(frozen=True, slots=True)
class PaperSearchResponse:
    query: str
    results: list[PaperSearchResult]
    searched_sources: list[str]
    skipped_sources: list[str]
    failed_sources: list[str]
    warnings: list[str]
```

---

8. 限流实现

rate_limit.py:

```
import asyncio
import time


class SourceRateGate:
    def __init__(self, min_interval_seconds: float) -> None:
        self._min_interval_seconds = min_interval_seconds
        self._lock = asyncio.Lock()
        self._last_request_at = 0.0

    async def wait(self) -> None:
        async with self._lock:
            now = time.monotonic()
            elapsed = now - self._last_request_at
            wait_seconds = self._min_interval_seconds - elapsed

            if wait_seconds > 0:
                await asyncio.sleep(wait_seconds)

            self._last_request_at = time.monotonic()
```

全局 source gate：

```
crossref_gate = SourceRateGate(settings.SOURCE_RATE_LIMIT_SECONDS)
datacite_gate = SourceRateGate(settings.SOURCE_RATE_LIMIT_SECONDS)
unpaywall_gate = SourceRateGate(settings.SOURCE_RATE_LIMIT_SECONDS)
arxiv_gate = SourceRateGate(settings.ARXIV_MIN_INTERVAL_SECONDS)
```

重点：

```
每个 source 一个 gate
每个 source 内部 lock 串行
外层允许不同 source 并发
```

---

9. HTTP 工具函数

http.py：

```
import asyncio
from typing import Optional

import httpx



async def get_json_with_retry(
    client: httpx.AsyncClient,
    url: str,
    *,
    params: dict,
    headers: Optional[dict] = None,
    retries: int = 2,
) -> dict:
    last_error: Optional[Exception] = None

    for attempt in range(retries + 1):
        try:
            response = await client.get(url, params=params, headers=headers)

            if response.status_code == 429:
                retry_after = response.headers.get("Retry-After")
                if retry_after and retry_after.isdigit():
                    await asyncio.sleep(int(retry_after))
                else:
                    await asyncio.sleep(2 ** attempt)
                continue
```

```
            response.raise_for_status()
            return response.json()

        except Exception as e:
            last_error = e
            if attempt < retries:
                await asyncio.sleep(2 ** attempt)

    raise RuntimeError(f"request failed after retries: {last_error}")
```

---

10. Crossref Source

sources/crossref_source.py:

```
import httpx

from chat.core.config.app_settings import settings
from chat.application.paper_search.http import get_json_with_retry
from chat.application.paper_search.models import PaperSearchResult, PaperSourceResponse
from chat.application.paper_search.rate_limit import SourceRateGate


class CrossrefSource:
    name = "crossref"

    def __init__(self, client: httpx.AsyncClient, gate: SourceRateGate) -> None:
        self._client = client
        self._gate = gate

    async def search(self, query: str, *, rows: int) -> PaperSourceResponse:
        await self._gate.wait()

        params = {
            "query.bibliographic": query,
            "rows": min(rows, 5),
            "select": (
                "DOI,title,author,issued,container-title,"
                "URL,abstract,published-print,published-online"
            ),
        }

        if settings.TOOL_CONTACT_EMAIL:
            params["mailto"] = settings.TOOL_CONTACT_EMAIL
```

```
        try:
            data = await get_json_with_retry(
                self._client,
                f"{settings.CROSSREF_BASE_URL}/works",
                params=params,
            )
        except Exception as e:
            return PaperSourceResponse(
                source_name=self.name,
                results=[],
                warnings=[f"crossref failed: {e}"],
                failed=True,
            )

        items = data.get("message", {}).get("items", [])
        return PaperSourceResponse(
            source_name=self.name,
            results=[_map_crossref_item(item) for item in items],
        )


def _map_crossref_item(item: dict) -> PaperSearchResult:
    title = _first(item.get("title")) or ""
    venue = _first(item.get("container-title"))
    doi = item.get("DOI")
    year = _extract_crossref_year(item)
    authors = _map_crossref_authors(item.get("author") or [])
    url = item.get("URL")

    return PaperSearchResult(
        title=title,
        authors=authors,
        year=year,
        abstract=item.get("abstract"),
        venue=venue,
        doi=doi.lower() if doi else None,
        url=url,
        source_urls=[url] if url else [],
        source_names=["crossref"],
        result_type="publisher_metadata",
        authority_score=0.9 if doi else 0.6,
        relevance_score=0.7,
    )


def _first(value: object) -> str | None:
    if isinstance(value, list) and value:
```

```
        first = value[0]
        return str(first) if first else None
    if isinstance(value, str):
        return value
    return None


def _extract_crossref_year(item: dict) -> int | None:
    for key in ("issued", "published-print", "published-online"):
        date_parts = item.get(key, {}).get("date-parts")
        if date_parts and date_parts[0]:
            return int(date_parts[0][0])
    return None


def _map_crossref_authors(authors: list[dict]) -> list[str]:
    result = []

    for author in authors:
        given = author.get("given")
        family = author.get("family")
        name = " ".join(part for part in [given, family] if part)

        if name:
            result.append(name)

    return result
```

---

11. arXiv Source

依赖:

```
uv add arxiv
```

sources/arxiv_source.py:

```
import asyncio

from arxiv import Client, Search, SortCriterion, SortOrder

from chat.core.config.app_settings import settings
from chat.application.paper_search.models import PaperSearchResult, PaperSourceResponse


class ArxivSource:
```

```
    name = "arxiv"

    def __init__(self) -> None:
        self._client = Client(
            page_size=settings.ARXIV_MAX_RESULTS,
            delay_seconds=settings.ARXIV_MIN_INTERVAL_SECONDS,
            num_retries=1,
        )

    async def search(self, query: str, *, rows: int) -> PaperSourceResponse:
        try:
            results = await asyncio.to_thread(self._search_sync, query, rows)
        except Exception as e:
            return PaperSourceResponse(
                source_name=self.name,
                results=[],
                warnings=[f"arxiv failed: {e}"],
                failed=True,
            )

        return PaperSourceResponse(
            source_name=self.name,
            results=results,
        )

    def _search_sync(self, query: str, rows: int) -> list[PaperSearchResult]:
        search = Search(
            query=f"all:{query}",
            max_results=min(rows, settings.ARXIV_MAX_RESULTS),
            sort_by=SortCriterion.SubmittedDate,
            sort_order=SortOrder.Descending,
        )

        results = []

        for item in self._client.results(search):
            published = item.published
            entry_id = item.entry_id
            pdf_url = item.pdf_url

            results.append(
                PaperSearchResult(
                    title=item.title or "",
                    authors=[str(author) for author in item.authors],
                    year=published.year if published else None,
                    abstract=item.summary,
                    arxiv_id=entry_id.rsplit("/", 1)[-1] if entry_id else None,
```

```
                    url=entry_id,
                    pdf_url=pdf_url,
                    source_urls=[url for url in [entry_id, pdf_url] if url],
                    source_names=["arxiv"],
                    publication_date=published.date().isoformat() if published else None,
                    result_type="preprint",
                    authority_score=0.55,
                    relevance_score=0.7,
                )
            )

        return results
```

说明：

```
不使用 xml.etree.ElementTree。
不手写 Atom XML 解析。
arxiv 包是同步 API，所以使用 asyncio.to_thread。
```

---

12. DataCite Source

sources/datacite_source.py：

```
import httpx

from chat.core.config.app_settings import settings
from chat.application.paper_search.http import get_json_with_retry
from chat.application.paper_search.models import PaperSearchResult, PaperSourceResponse
from chat.application.paper_search.rate_limit import SourceRateGate


class DataCiteSource:
    name = "datacite"

    def __init__(self, client: httpx.AsyncClient, gate: SourceRateGate) -> None:
        self._client = client
        self._gate = gate

    async def search(self, query: str, *, rows: int) -> PaperSourceResponse:
        await self._gate.wait()

        params = {
            "query": query,
            "page[size]": min(rows, 5),
        }
```

```
        try:
            data = await get_json_with_retry(
                self._client,
                f"{settings.DATACITE_BASE_URL}/dois",
                params=params,
            )
        except Exception as e:
            return PaperSourceResponse(
                source_name=self.name,
                results=[],
                warnings=[f"datacite failed: {e}"],
                failed=True,
            )

        items = data.get("data", [])
        return PaperSourceResponse(
            source_name=self.name,
            results=[_map_datacite_item(item) for item in items],
        )


def _map_datacite_item(item: dict) -> PaperSearchResult:
    attrs = item.get("attributes") or {}

    title = ""
    titles = attrs.get("titles") or []
    if titles:
        title = titles[0].get("title") or ""

    doi = attrs.get("doi")
    url = attrs.get("url")
    creators = attrs.get("creators") or []
    descriptions = attrs.get("descriptions") or []
    types = attrs.get("types") or {}

    abstract = None
    if descriptions:
        abstract = descriptions[0].get("description")

    return PaperSearchResult(
        title=title,
        authors=[creator.get("name") for creator in creators if creator.get("name")],
        year=attrs.get("publicationYear"),
        abstract=abstract,
        doi=doi.lower() if doi else None,
        url=url,
```

```
        source_urls=[url] if url else [],
        source_names=["datacite"],
        result_type=types.get("resourceTypeGeneral") or "research_object",
        authority_score=0.75 if doi else 0.5,
        relevance_score=0.6,
    )
```

---

13. Unpaywall Source

sources/unpaywall_source.py:

```
from dataclasses import replace

import httpx

from chat.core.config.app_settings import settings
from chat.application.paper_search.http import get_json_with_retry
from chat.application.paper_search.models import PaperSearchResult
from chat.application.paper_search.rate_limit import SourceRateGate


class UnpaywallSource:
    name = "unpaywall"

    def __init__(self, client: httpx.AsyncClient, gate: SourceRateGate) -> None:
        self._client = client
        self._gate = gate

    async def enrich(self, result: PaperSearchResult) -> tuple[PaperSearchResult, list[str]]:
        if not settings.TOOL_CONTACT_EMAIL:
            return result, ["unpaywall skipped: TOOL_CONTACT_EMAIL is missing"]

        if not result.doi:
            return result, []

        await self._gate.wait()

        try:
            data = await get_json_with_retry(
                self._client,
                f"{settings.UNPAYWALL_BASE_URL}/v2/{result.doi}",
                params={"email": settings.TOOL_CONTACT_EMAIL},
            )
        except Exception as e:
            return result, [f"unpaywall failed for DOI {result.doi}: {e}"]
```

```
        best_location = data.get("best_oa_location") or {}

        pdf_url = best_location.get("url_for_pdf") or result.pdf_url
        landing_url = best_location.get("url") or result.url

        source_urls = _merge_unique(
            result.source_urls,
            [url for url in [landing_url, pdf_url] if url],
        )

        return replace(
            result,
            pdf_url=pdf_url,
            url=landing_url,
            is_open_access=data.get("is_oa"),
            source_names=_merge_unique(result.source_names, ["unpaywall"]),
            source_urls=source_urls,
        ), []


async def enrich_with_unpaywall_serial(
    results: list[PaperSearchResult],
    source: UnpaywallSource,
    *,
    limit: int = 5,
) -> tuple[list[PaperSearchResult], list[str]]:
    enriched = []
    warnings = []

    for result in results[:limit]:
        updated, item_warnings = await source.enrich(result)
        enriched.append(updated)
        warnings.extend(item_warnings)

    enriched.extend(results[limit:])
    return enriched, warnings


def _merge_unique(left: list[str], right: list[str]) -> list[str]:
    result = []
    seen = set()

    for item in [*left, *right]:
        if item and item not in seen:
            result.append(item)
            seen.add(item)
```

```
    return result
```

---

14. Query 处理

query.py：

```
def normalize_query(query: str) -> str:
    return " ".join(query.strip().split())
```

不做：

```
自动翻译
自动 query expansion
自动多 query 并发
```

把"中文 query 要翻译成英文 academic query"的责任写进 tool description。

---

15. 去重

dedup.py：

```
from dataclasses import replace
import re

from chat.application.paper_search.models import PaperSearchResult


def deduplicate_papers(results: list[PaperSearchResult]) -> list[PaperSearchResult]:
    merged: dict[str, PaperSearchResult] = {}

    for result in results:
        key = paper_key(result)
        existing = merged.get(key)

        if existing is None:
            merged[key] = result
        else:
            merged[key] = merge_paper(existing, result)

    return list(merged.values())


def paper_key(result: PaperSearchResult) -> str:
```

```
    if result.doi:
        return f"doi:{result.doi.lower()}"

    if result.arxiv_id:
        return f"arxiv:{result.arxiv_id.lower()}"

    title = normalize_title(result.title)
    first_author = result.authors[0].lower() if result.authors else ""
    year = result.year or ""

    return f"title:{title}|author:{first_author}|year:{year}"


def normalize_title(title: str) -> str:
    text = title.lower()
    text = re.sub(r"[^a-z0-9]+", " ", text)
    return " ".join(text.split())


def merge_paper(a: PaperSearchResult, b: PaperSearchResult) -> PaperSearchResult:
    return replace(
        a,
        title=a.title or b.title,
        authors=a.authors or b.authors,
        year=a.year or b.year,
        abstract=a.abstract or b.abstract,
        venue=a.venue or b.venue,
        doi=a.doi or b.doi,
        arxiv_id=a.arxiv_id or b.arxiv_id,
        url=a.url or b.url,
        pdf_url=a.pdf_url or b.pdf_url,
        source_urls=_merge_unique(a.source_urls, b.source_urls),
        source_names=_merge_unique(a.source_names, b.source_names),
        is_open_access=a.is_open_access if a.is_open_access is not None else b.is_open_access,
        authority_score=max(a.authority_score, b.authority_score),
        relevance_score=max(a.relevance_score, b.relevance_score),
    )


def _merge_unique(left: list[str], right: list[str]) -> list[str]:
    result = []
    seen = set()

    for item in [*left, *right]:
        if item and item not in seen:
            result.append(item)
            seen.add(item)
```

```
    return result
```

---

16. 排序

ranking.py:

```
from chat.application.paper_search.models import PaperSearchResult


def rank_papers(results: list[PaperSearchResult]) -> list[PaperSearchResult]:
    return sorted(
        results,
        key=lambda item: (
            _source_consensus_score(item),
            item.authority_score,
            _recency_score(item),
            item.relevance_score,
        ),
        reverse=True,
    )


def _source_consensus_score(result: PaperSearchResult) -> float:
    return min(len(set(result.source_names)) * 0.2, 0.6)


def _recency_score(result: PaperSearchResult) -> float:
    if not result.year:
        return 0.0

    if result.year >= 2024:
        return 0.3

    if result.year >= 2020:
        return 0.2

    return 0.1
```

---

17. Service 聚合逻辑

service.py:

```
import asyncio

import httpx

from chat.core.config.app_settings import settings
from chat.application.paper_search.dedup import deduplicate_papers
from chat.application.paper_search.models import PaperSearchRequest, PaperSearchResponse
from chat.application.paper_search.query import normalize_query
from chat.application.paper_search.ranking import rank_papers
from chat.application.paper_search.rate_limit import SourceRateGate
from chat.application.paper_search.sources.arxiv_source import ArxivSource
from chat.application.paper_search.sources.crossref_source import CrossrefSource
from chat.application.paper_search.sources.datacite_source import DataCiteSource
from chat.application.paper_search.sources.unpaywall_source import (
    UnpaywallSource,
    enrich_with_unpaywall_serial,
)


_crossref_gate = SourceRateGate(settings.SOURCE_RATE_LIMIT_SECONDS)
_datacite_gate = SourceRateGate(settings.SOURCE_RATE_LIMIT_SECONDS)
_unpaywall_gate = SourceRateGate(settings.SOURCE_RATE_LIMIT_SECONDS)


class PaperSearchService:
    async def search(self, request: PaperSearchRequest) -> PaperSearchResponse:
        query = normalize_query(request.query)
        max_results = min(request.max_results, settings.PAPER_SEARCH_MAX_RESULTS)

        async with httpx.AsyncClient(timeout=settings.PAPER_SEARCH_TIMEOUT_SECONDS) as client:
            tasks = []

            if settings.PAPER_SEARCH_ENABLE_CROSSREF:
                crossref = CrossrefSource(client, _crossref_gate)
                tasks.append(crossref.search(query, rows=5))

            if settings.PAPER_SEARCH_ENABLE_ARXIV:
                arxiv = ArxivSource()
                tasks.append(arxiv.search(query, rows=5))

            if settings.PAPER_SEARCH_ENABLE_DATACITE:
                datacite = DataCiteSource(client, _datacite_gate)
                tasks.append(datacite.search(query, rows=5))

            source_responses = await asyncio.gather(*tasks, return_exceptions=True)
```

```
    searched_sources: list[str] = []
    failed_sources: list[str] = []
    warnings: list[str] = []
    raw_results = []

    for response in source_responses:
        if isinstance(response, Exception):
            warnings.append(f"source task failed: {response}")
            continue

        searched_sources.append(response.source_name)
        warnings.extend(response.warnings)

        if response.failed:
            failed_sources.append(response.source_name)

        raw_results.extend(response.results)

    if not raw_results and searched_sources and len(failed_sources) == len(searched_sources):
        raise RuntimeError("all paper search sources failed")

    merged = deduplicate_papers(raw_results)
    ranked = rank_papers(merged)

    skipped_sources = []

    if settings.PAPER_SEARCH_ENABLE_UNPAYWALL:
        if settings.TOOL_CONTACT_EMAIL:
            unpaywall = UnpaywallSource(client, _unpaywall_gate)
            ranked, unpaywall_warnings = await enrich_with_unpaywall_serial(
                ranked[:max_results],
                unpaywall,
                limit=5,
            )
            warnings.extend(unpaywall_warnings)
        else:
            skipped_sources.append("unpaywall")
            warnings.append("unpaywall skipped: TOOL_CONTACT_EMAIL is missing")

    return PaperSearchResponse(
        query=query,
        results=ranked[:max_results],
        searched_sources=searched_sources,
        skipped_sources=skipped_sources,
        failed_sources=failed_sources,
```

```
                warnings=warnings,
            )
```

---

18. Tool Schema

paper_search_tool.py:

```
_TOOL_SCHEMA = {
    "type": "object",
    "properties": {
        "query": {
            "type": "string",
            "description": (
                "Concise English academic search keywords. "
                "For non-English user requests, translate the concept into English before calling. "
                "Example: use 'deep learning' instead of '深度学习'."
            ),
        },
        "max_results": {
            "type": "integer",
            "default": 8,
            "minimum": 1,
            "maximum": 10,
        },
    },
    "required": ["query"],
    "additionalProperties": False,
}
```

---

19. Tool Description

```
_TOOL_DESCRIPTION = (
    "Searches free and open scholarly sources for academic papers using Crossref, arXiv, DataCite, "
    "and optionally Unpaywall. Use this tool for paper discovery, DOI-backed publication metadata, "
    "preprints, datasets, research objects, and open-access paper links.\n\n"
    "This tool intentionally avoids paid commercial databases and freemium quota-based APIs. "
    "It does not use OpenAlex API or Semantic Scholar API by default. "
    "Do not rely on arXiv alone. arXiv is a preprint source and has strict request limits. "
    "For non-English user queries, use concise English academic keywords when calling this tool. "
    "The tool may return partial results with source warnings if one source is rate-limited or
unavailable."
)
```

---

20. Tool Result 格式

```
[Tool Result] paper_search

Query: deep learning

Sources searched:
- crossref
- arxiv
- datacite

Sources skipped:
- unpaywall: TOOL_CONTACT_EMAIL is missing

Sources failed:
- none

Warnings:
- unpaywall skipped: TOOL_CONTACT_EMAIL is missing

Results:
[1]
- title:
- authors:
- year:
- venue:
- doi:
- arxiv_id:
- url:
- pdf_url:
- source_names:
- source_urls:
- result_type:
- publication_date:
- is_open_access:
- abstract:

Assistant instructions:
- Prefer DOI-backed and publisher-backed records from Crossref or DataCite when available.
- Treat arXiv results as preprints unless there is DOI or publisher metadata.
- If one source failed, was skipped, or was rate-limited, do not claim that no papers exist.
- Summarize source coverage and warnings.
- If the user asks for full text, prefer OA links from Unpaywall or arXiv PDF when available.
- If no result is found and the original user query was not English, retry once with concise English
academic keywords.
```

21. 失败策略

```
单个 source 失败：
- 返回其他 source 结果
- warnings 记录失败

Unpaywall 缺 email：
- skipped_sources 记录 unpaywall
- warnings 记录 TOOL_CONTACT_EMAIL missing
- 不报错

0 结果：
- 不是 Tool Error
- 提醒模型英文 academic query 重试

所有主 source 都失败：
- Tool Error
```

主 source 指：

```
Crossref
arXiv
DataCite
```

Unpaywall 不是主 source。

---

22. 缓存策略

第一版可以做轻量 TTL cache：

```
key = source_name + normalized_query + rows
ttl = PAPER_SEARCH_CACHE_TTL_SECONDS
```

缓存位置：

```
内存缓存即可
```

不做：

```
Redis cache
DB cache
持久化 cache
复杂 cache invalidation
```

缓存目的：

```
减少短时间重复查询
降低限流风险
```

---

23. 注册策略

新增：

```
paper_search_tool.py
```

并在工具注册处注册：

```
tool_registry.register(PaperSearchTool())
```

建议对模型主推：

```
paper_search
```

而不是直接使用：

```
arxiv_search
```

`arxiv_search` 可以保留，但应该变成低层工具或兼容工具。

---

24. 与 arxiv_search 的关系

```
paper_search 是主入口。
arxiv_search 保留。
模型搜索论文时优先 paper_search。
只有明确要求 arXiv-only 时，才用 arxiv_search。
```

建议在 `arxiv_search` description 里补充：

```
For general paper discovery, prefer paper_search because it combines Crossref, arXiv, DataCite, and
Unpaywall. Use arxiv_search only when the user explicitly wants arXiv preprints or arXiv IDs.
```

---

25. 测试用例

25.1 正常查询

```
query="deep learning"
```

期望：

```
调用 Crossref / arXiv / DataCite
返回 searched_sources
不依赖单一 arXiv
```

25.2 中文 query

用户：

```
找一些深度学习最新论文
```

期望模型调用：

```
{
  "query": "deep learning",
  "max_results": 8
}
```

不应调用：

```
{
  "query": "深度学习"
}
```

25.3 arXiv 失败

模拟 arXiv timeout / 429。

期望：

```
paper_search 仍返回 Crossref / DataCite
warnings 包含 arxiv failed
不是 Tool Error
```

25.4 Crossref 失败

模拟 Crossref timeout / 429。

期望：

```
paper_search 仍返回 arXiv / DataCite
warnings 包含 crossref failed
不是 Tool Error
```

25.5 DataCite 失败

模拟 DataCite timeout / 429。

期望：

```
paper_search 仍返回 Crossref / arXiv
warnings 包含 datacite failed
不是 Tool Error
```

25.6 TOOL_CONTACT_EMAIL 缺失

期望：

```
Unpaywall skipped
warnings 包含 TOOL_CONTACT_EMAIL is missing
不是 Tool Error
```

25.7 Unpaywall 串行

有多个 DOI 时：

```
最多处理前 5 个 DOI
按 for-loop 串行处理
不 asyncio.gather
```

25.8 DOI 去重

Crossref 和 DataCite 返回同 DOI。

期望：

```
合并为一个结果
source_names 包含 crossref / datacite
```

25.9 只有 arXiv

期望：

```
result_type=preprint
```

```
Assistant instructions 提醒按 preprint 处理
```

25.10 所有主 source 失败

期望:

```
Tool Error
```

---

26. E2E 观测 prompt

```
找一些 deep learning 最新论文，优先给权威来源。
```

期望:

```
tools: paper_search
query: deep learning
sources_searched: crossref, arxiv, datacite
```

```
找一些深度学习最新论文。
```

期望:

```
tools: paper_search
query: deep learning
不要直接注入中文 query
```

```
找一下 attention is all you need 的论文来源和可访问链接。
```

期望:

```
tools: paper_search
Crossref / DataCite / arXiv 至少部分命中
如果有 DOI，尝试 Unpaywall
```

---

27. Codex 最终执行提示词

```
请实现 paper_search v1。
```

最终定稿：
paper_search 只使用以下免费开放来源：
1. Crossref
2. arXiv
3. DataCite
4. Unpaywall

不要接入：
- OpenAlex
- Semantic Scholar
- PubMed
- Europe PMC
- DOAJ
- OpenCitations
- Scopus
- Web of Science
- Dimensions
- Elsevier / Springer / IEEE / ACM 付费 API

核心目标：
避免只依赖 arXiv 导致论文搜索不稳定。
paper_search 是通用论文搜索主入口。
arxiv_search 保留，但一般论文搜索优先使用 paper_search。

依赖：
uv add arxiv

实现原则：
1. 尽量使用 SDK / 成熟客户端。
2. arXiv 使用 arxiv Python 包，不手写 Atom XML 解析。
3. Crossref / DataCite / Unpaywall 使用官方 REST API 薄 adapter。
4. 不做复杂二次解析。
5. 只解析必要稳定字段。
6. 只允许多来源并发，不允许单一来源内部并发。
7. 不为了性能大开高并发，防止限流。

并发限制：
1. Crossref / arXiv / DataCite 可以并发。
2. 每个 source search 只发 1 个请求。
3. Crossref 不做多页并发。
4. arXiv 不做多页并发。
5. DataCite 不做多页并发。
6. Unpaywall 不做多 DOI 并发。
7. Unpaywall 串行处理前 5 个 DOI。
8. arXiv 使用 delay_seconds=ARXIV_MIN_INTERVAL_SECONDS，默认 3 秒。
9. Crossref / DataCite / Unpaywall 使用 SourceRateGate，默认 1 秒。
10. 支持 429 Retry-After 和指数退避。

新增目录：

```
chat/application/paper_search/
  __init__.py
  models.py
  service.py
  formatting.py
  query.py
  dedup.py
  ranking.py
  cache.py
  rate_limit.py
  http.py
  sources/
    __init__.py
    crossref_source.py
    arxiv_source.py
    datacite_source.py
    unpaywall_source.py
```

新增工具：

```
chat/application/tools/paper_search_tool.py
```

配置：

```
PAPER_SEARCH_TIMEOUT_SECONDS = 12.0
PAPER_SEARCH_MAX_RESULTS = 8
PAPER_SEARCH_CACHE_TTL_SECONDS = 3600

PAPER_SEARCH_ENABLE_CROSSREF = True
PAPER_SEARCH_ENABLE_ARXIV = True
PAPER_SEARCH_ENABLE_DATACITE = True
PAPER_SEARCH_ENABLE_UNPAYWALL = True

CROSSREF_BASE_URL = "https://api.crossref.org"
DATACITE_BASE_URL = "https://api.datacite.org"
UNPAYWALL_BASE_URL = "https://api.unpaywall.org"

SOURCE_RATE_LIMIT_SECONDS = 1.0
ARXIV_MIN_INTERVAL_SECONDS = 3.0
ARXIV_MAX_RESULTS = 5

TOOL_CONTACT_EMAIL: Optional[str] = None
```

数据模型：

```
PaperSearchRequest
PaperSearchResult
PaperSourceResponse
```

PaperSearchResponse

Source 实现：
1. CrossrefSource
- GET {CROSSREF_BASE_URL}/works
- query.bibliographic=query
- rows=min(rows, 5)
- select=DOI,title,author,issued,container-title,URL,abstract,published-print,published-online
- mailto=TOOL_CONTACT_EMAIL if present
- result_type=publisher_metadata

2. ArxivSource
- 使用 arxiv.Client
- delay_seconds=ARXIV_MIN_INTERVAL_SECONDS
- page_size=ARXIV_MAX_RESULTS
- Search(query=f"all:{query}", max_results=min(rows, ARXIV_MAX_RESULTS), sort_by=SubmittedDate, sort_order=Descending)
- 使用 asyncio.to_thread 包同步调用
- result_type=preprint

3. DataCiteSource
- GET {DATACITE_BASE_URL}/dois
- query=query
- page[size]=min(rows, 5)
- result_type=attributes.types.resourceTypeGeneral or research_object

4. UnpaywallSource
- 只在 TOOL_CONTACT_EMAIL 存在时启用
- GET {UNPAYWALL_BASE_URL}/v2/{doi}?email={TOOL_CONTACT_EMAIL}
- 只解析 is_oa, best_oa_location.url, best_oa_location.url_for_pdf
- 串行处理前 5 个 DOI

Dedup：
优先：
1. DOI
2. arXiv ID
3. normalized title + first author + year

Ranking：
综合：
- source consensus
- authority_score
- recency_score
- relevance_score

Tool schema：
query: string, required

max_results: integer, default 8, min 1, max 10

query description:
Concise English academic search keywords. For non-English user requests, translate the concept into English before calling. Example: use "deep learning" instead of "深度学习".

Tool description:
Searches free and open scholarly sources for academic papers using Crossref, arXiv, DataCite, and optionally Unpaywall. Use this tool for paper discovery, DOI-backed publication metadata, preprints, datasets, research objects, and open-access paper links.

This tool intentionally avoids paid commercial databases and freemium quota-based APIs. It does not use OpenAlex API or Semantic Scholar API by default. Do not rely on arXiv alone. arXiv is a preprint source and has strict request limits. For non-English user queries, use concise English academic keywords when calling this tool. The tool may return partial results with source warnings if one source is rate-limited or unavailable.

Tool Result:
必须包含：
- Query
- Sources searched
- Sources skipped
- Sources failed
- Warnings
- Results
- Assistant instructions

Assistant instructions:
- Prefer DOI-backed and publisher-backed records from Crossref or DataCite when available.
- Treat arXiv results as preprints unless there is DOI or publisher metadata.
- If one source failed, was skipped, or was rate-limited, do not claim that no papers exist.
- Summarize source coverage and warnings.
- If the user asks for full text, prefer OA links from Unpaywall or arXiv PDF when available.
- If no result is found and the original user query was not English, retry once with concise English academic keywords.

失败策略：
- 单个 source 失败只 warning。
- 0 结果不是 Tool Error。
- Unpaywall 缺 TOOL_CONTACT_EMAIL 时 skipped + warning。
- 所有主 source 失败才 Tool Error。
- 主 source 是 Crossref / arXiv / DataCite。

测试：
1. query="deep learning" 调用 Crossref / arXiv / DataCite。
2. 用户中文“深度学习”时模型应注入 query="deep learning"。
3. arXiv 失败不影响 Crossref / DataCite。

4. Crossref 失败不影响 arXiv / DataCite。
5. DataCite 失败不影响 Crossref / arXiv。
6. Unpaywall 缺 email 时 skipped + warning。
7. Unpaywall 串行处理前 5 个 DOI。
8. DOI 重复结果合并。
9. 只有 arXiv 来源时 result_type=preprint。
10. 所有主 source 失败才 Tool Error。
11. Tool Result 不超过 TOOL_RESULT_MAX_CHARS。
12. arxiv_source.py 不出现 xml.etree.ElementTree 解析逻辑。
13. 不出现 OpenAlex / Semantic Scholar / PubMed / Europe PMC 相关实现。

---

28. 最终结论

```
paper_search v1 = Crossref + arXiv + DataCite + Unpaywall
```

并发策略:

```
只做多来源并发;
单一来源内部永远串行、低频、限速。
```

实现策略:

```
arXiv 用 arxiv 包;
Crossref / DataCite / Unpaywall 用官方 REST API 薄 adapter;
不做复杂二次解析。
```

模型策略:

```
论文搜索优先 paper_search;
只有用户明确要 arXiv 时才用 arxiv_search;
中文论文查询先转英文 academic query。
```